1팀

정신우: Project Manager
손영욱: Data Engineer (R-CNN)
엄지영: Model Architect
김범진: Experimentation Lead

01 Project Manager

02 Data Engineer (R-CNN)

03 Model Architect

04 Experimentation Lead

01 Project Manager

02 Data Engineer (R-CNN)

03 Model Architect

04 Experimentation Lead

데일리 브리핑

09시

아침 브리핑

엄지영 오전 09:09
before_aug_sampling으로 학습해 봤습니다. aug_sampling_data_final 때보다 확실히 mAP가 낮게 나왔어요

김범진 오전 09:12
파라미터들 중에서 성능이 제일 좋은걸 고르면 돼서 데이터 수량 및 에포크 낮춰서 돌렸고 결과 노션에 올렸습니다.
https://aged-bee-a92.notion.site/yolo26-hyperparameter-33330de5ce14805d909fc2cb2e798227

손영욱 오전 09:07
저는 샘플링까지 진행한거 추가하고 학습중이고

손영욱 오전 09:08
하면서 멘토링때 했던 알약모양 나눠서 학습할 수 있도록

데이터분할해보려합니다

14시

중간 브리핑

정신우 오후 02:35
1. 증강이 좀 과했다
2. 해상도가 높였을 때의 결과를 봐야한다.
3. 라벨링 품질

김범진 오후 02:35
흠 저도 augmentation 건들어서

몇 픽셀 차이로 IOU 95 점수가 낮게 나오는 것 같긴한데

손영욱 오후 02:35
3은 박스가 약보다 좀 큰거 빼면

문제는 없을거같긴합니다

18시

저녁 브리핑(선택)

손영욱 오후 06:34
원형 0,9661
장방형 0.9485
타원형0.9569
네요

손영욱 오후 06:35
50-95입니다

정신우 오후 06:35
RCNN추론해서 score 내봐도 좋을 것 같아요!

손영욱 오후 06:35
각 모델 checkpoint 도올려볼게요

통합해서 스코어 내려면

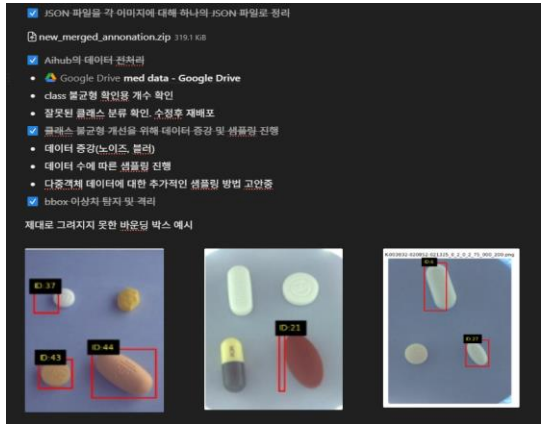
음.. 어떻게해야할까요

정신우 오후 06:36
제 생각에는 각 이미지마다 세 개의 체크포인트를 다 돌리고, nms이용해서 알약 하나당 하나의 bbox만 남게 구현하면 좋을 것 같긴해요

노션 관리

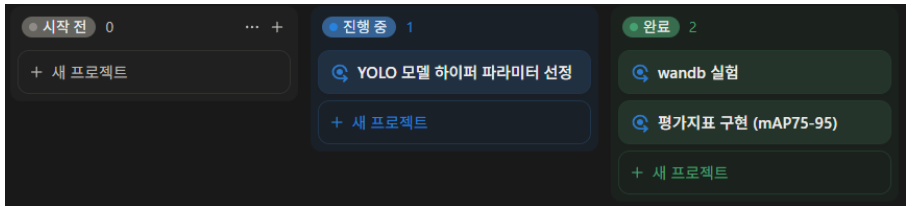
정신우

손영욱



엄지영

김범진



데이터 관리

codeit_1팀

- checkpoints
- med_data
- runs
- ppt 템플릿
- pptSON.pptx

med_data

- 데이터 split
- 이상한 bbox 삭제 후 데이터셋
- aug_data.zip
- aug_sampling_bbox이상항목
- aug_sampling_data_final.zip
- before_aug_sampling.zip
- class_count.txt
- div_by_shape.zip
- new_merged_annonation.zip

checkpoints

- Resnet50 faster r-cnn
- yolo26m_데이터셋별
- yolo26m_aug sampling data_Tune
- yolo26m_CLAHE
- yolo26m_train
- yolo26n
- yolov7 vs yolo26m

01 Project Manager

02 Data Engineer (R-CNN)

03 Model Architect

04 Experimentation Lead

데이터 산점도

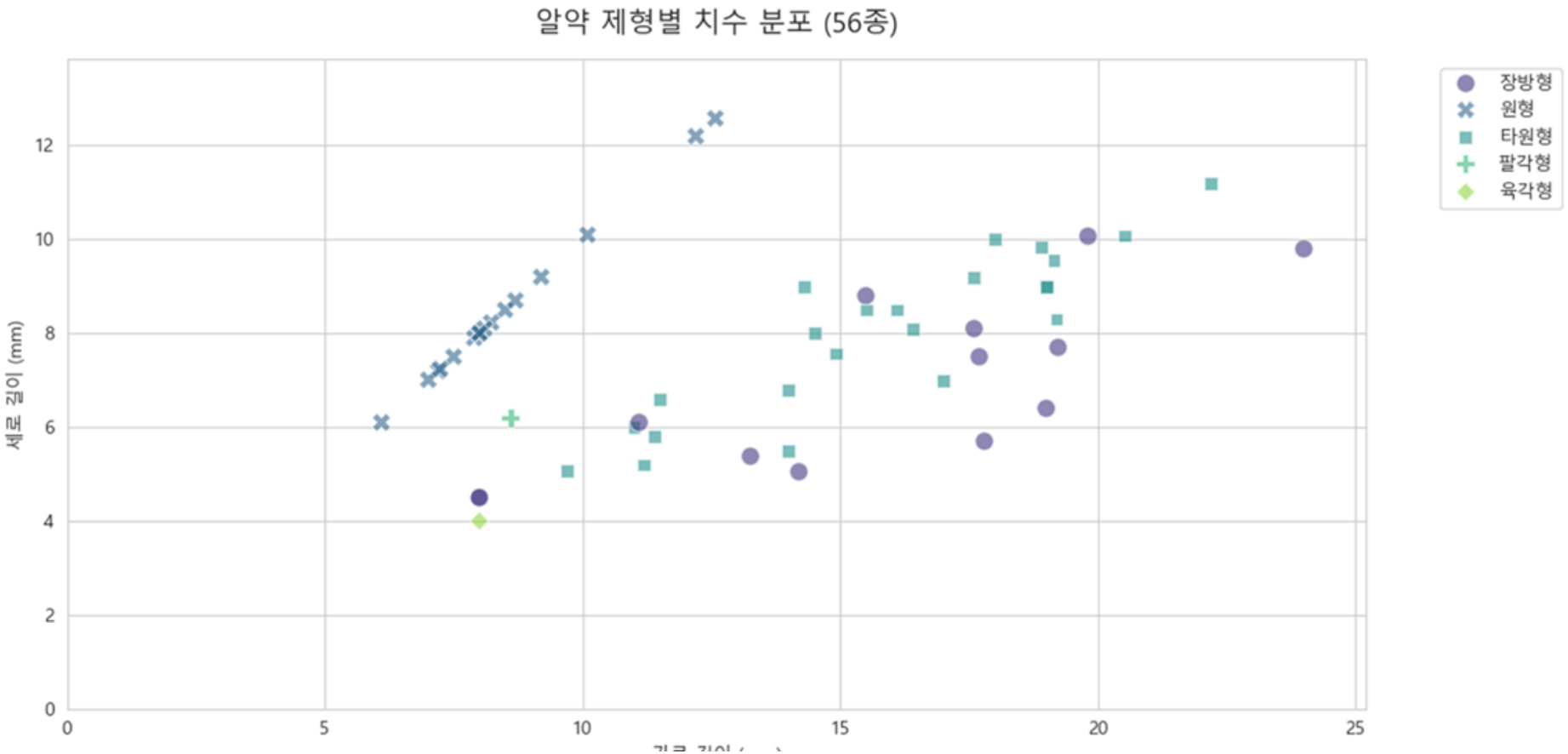
Kaggle 데이터 기반

232개의 이미지 데이터

총 56개의 알약 클래스 검출

팔각형과 육각형 클래스는 1개

클래스간 다양한 크기로 분포



데이터 문제 1) 부족한 데이터셋

기본 Kaggle 데이터셋의 한계

- **56개의 알약 클래스**
- **객체 수에 비해 부족한 학습 이미지 데이터**

train_images

232 files

추가 데이터셋 확보

- **Aihub 에서 TL_2를 제외한 추가적인 다중객체 이미지를 확보.**
- **필요하지 않은 클래스로 구성된 데이터는 제거**
- **약 8000장의 이미지 데이터를 확보**

크기:	13.5GB (14,503,650,660 바이트)
디스크 할당 크기:	13.5GB (14,520,143,872 바이트)
내용:	파일 8,039, 폴더 0

데이터셋 규모를 확장시킴으로서 일반화 성능 향상

데이터 문제 2) 분산된 Annotation 구조

기존 Annotation 구조

- 이미지 하나에 여러개의 객체가 들어가 있는 경우 객체별 정보가 다 다른 JSON 파일로 저장되어 있음.
- 해당 구조는 하나의 이미지 처리를 위해 매번 여러개의 JSON 파일을 호출/대조해야 함.
- 그로 인해 I/O 오버헤드 및 코드 구현에서의 복잡성이 예상되어 이미지 하나에 하나의 JSON 파일 구조로 재설계

기존 class 번호 구조

- 대부분의 모델에서는 class 번호를 0번부터 순서대로 지정하는 방식을 주로 사용한다.
- 현재 class 번호로 이용할 category_id가 새로 얻은 데이터는 모두 0번, 기존 kaggle 데이터는 넘버링 숫자로 들어가 있다.
- 모든 약의 category_id를 재매핑하여 0~55까지의 번호로 지정.

개발 효율 향상과 학습 속도를 최적화하는 결과 기대

데이터 문제 2) 분산된 Annotation 구조


```
"annotations": [
    {
        "area": 142800,
        "iscrowd": 0,
        "bbox": [
            109,
            580,
            400,
            357
        ],
        "category_id": 3832,
        "ignore": 0,
        "segmentation": [],
        "id": 1056,
        "image_id": 281
    }
```

```
"annotations": [
    {
        "area": 34965,
        "iscrowd": 0,
        "bbox": [
            339,
            171,
            185,
            189
        ],
        "category_id": 3351,
        "ignore": 0,
        "segmentation": [],
        "id": 1050,
        "image_id": 281
    }
],
```

```
"annotations": [
    {
        "area": 34965,
        "iscrowd": 0,
        "bbox": [
            339,
            171,
            185,
            189
        ],
        "category_id": 2,
        "ignore": 0,
        "segmentation": [],
        "id": 1,
        "image_id": 281
    },
    {
        "area": 142800,
        "iscrowd": 0,
        "bbox": [
            109,
            580,
            400,
            357
        ],
        "category_id": 6,
        "ignore": 0,
        "segmentation": [],
        "id": 2,
        "image_id": 281
    },
    {
        "area": 37245,
        "iscrowd": 0,
```

데이터 문제 3) 데이터 검증 및 노이즈 제거

BBox 정의 형식 확인

- 데이터셋이 COCO 형식이므로 BBox 정보가 [x, y, w, h] 구조로 되어있는지 확인.
- 3개의 요소만 있거나 5개의 요소가 있는 경우가 있었고 BBox 정보가 아예 없는 경우도 있었음.

```python
# 키 존재 여부 및 비어있는지 확인
if 'annotations' not in data or not data['annotations']:
    is_valid = False
    reason = "객체 데이터(annotations) 없음"
else:
    for ann in data['annotations']:
        if 'bbox' not in ann:
            is_valid = False
            reason = "bbox 키 누락"
            break

        bbox = ann['bbox']

        if not isinstance(bbox, list) or len(bbox) != 4:
            is_valid = False
            reason = f"bbox 좌표 개수 오류 (현재 {len(bbox) if isinstance(bbox, list) else 'list 아님'}개)"
            break

        x, y, w, h = bbox
        if w <= 0 or h <= 0:
            is_valid = False
            reason = f"유효하지 않은 크기 (w:{w}, h:{h})"
            break

# 문제 파일 격리
if not is_valid:
    print(f"격리 대상: {j_file} | 사유: {reason}")
    shutil.move(json_path, os.path.join(backup_dir, j_file))
    invalid_count += 1
```

데이터 문제 3) 데이터 검증 및 노이즈 제거

BBox 위치 정확도 확인

1. Grayscale 및 이진화

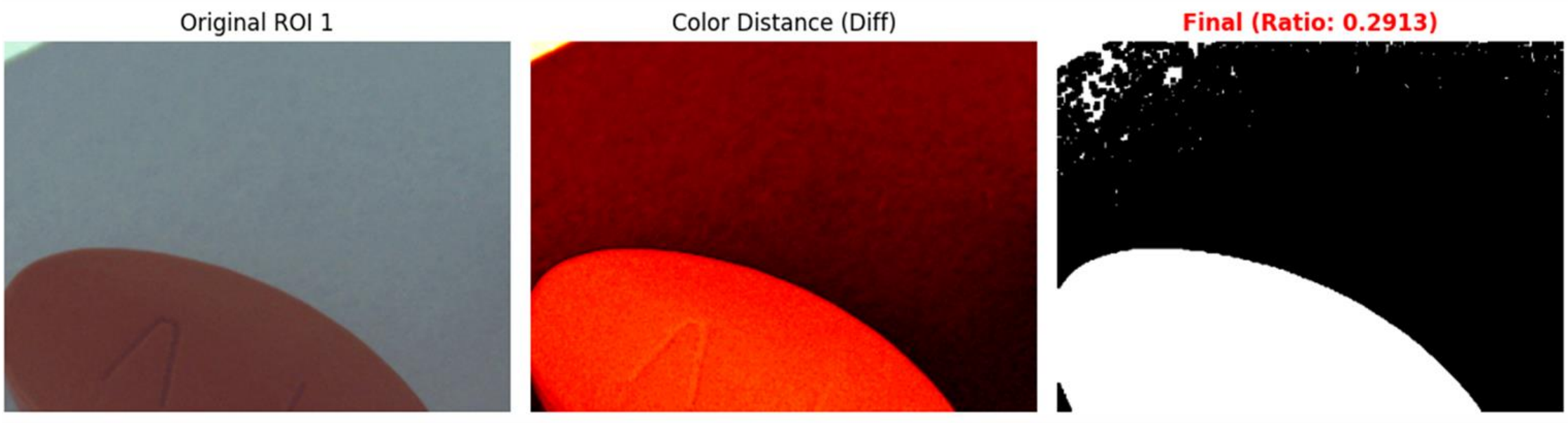
- 이미지를 graysacle 화 하여 이진화를 진행해 배경과 알약을 구분하는 방법 고안.
- 하지만 그림자 간섭, 배경과 알약의 색이 유사한 경우, 알약이 2개의 색상으로 이루어진 캡슐형 알약일 경우 등 알약과 배경의 이진화가 제대로 이뤄지지 않는 경우 발생

2. HSV 변환 및 통계적 임계값 활용

- 색상, 채도, 명도로 이뤄진 HSV 값으은 조명에 강해 색상에 대해서 집중하기 쉬움
- BBox 테두리 영역의 H값을 이용하여 통계적 접근을 통해 배경 색을 감지. 여기서 테두리에 약이 걸쳐져 있을 경우를 위해 통계적 기법을 사용함.
- 그후 배경색과 일정 수준 이상 차이나는 색상을 알약으로 판단하여 이진화.

데이터 문제 3) 데이터 검증 및 노이즈 제거

BBox 위치 정확도 확인



하지만 배경이 단색이어서 가능한 기법이기도 하며 해당 알고리즘도 배경색과 알약의 색이 거의 비슷한 경우 제대로 잡아내지 못함

결국 최종 이상치는 직접 BBox 시각화를 통해 검출하였음.

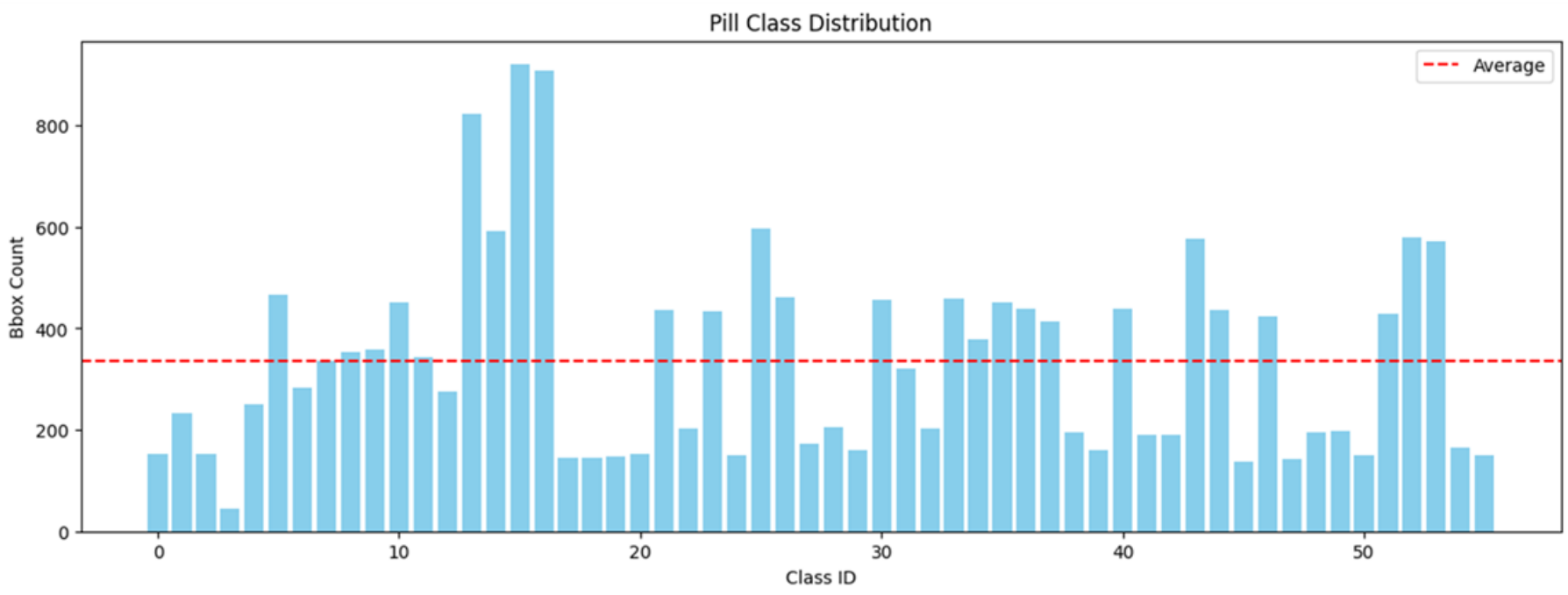
데이터 문제 4) 클래스 불균형

클래스 불균형 확인.

최대 빈도 클래스 : 850개

최소 빈도 클래스 : 45개

20배의 차이가 나타남으로
소수 클래스의 Recall 이
떨어질 우려가 있음.



데이터 문제 4) 클래스 불균형

해결 전략 1) 소수 클래스 데이터 증강

원본

블러

노이즈

데이터 문제 4) 클래스 불균형

해결 전략 2) 다중 객체 기반 전략적 샘플링

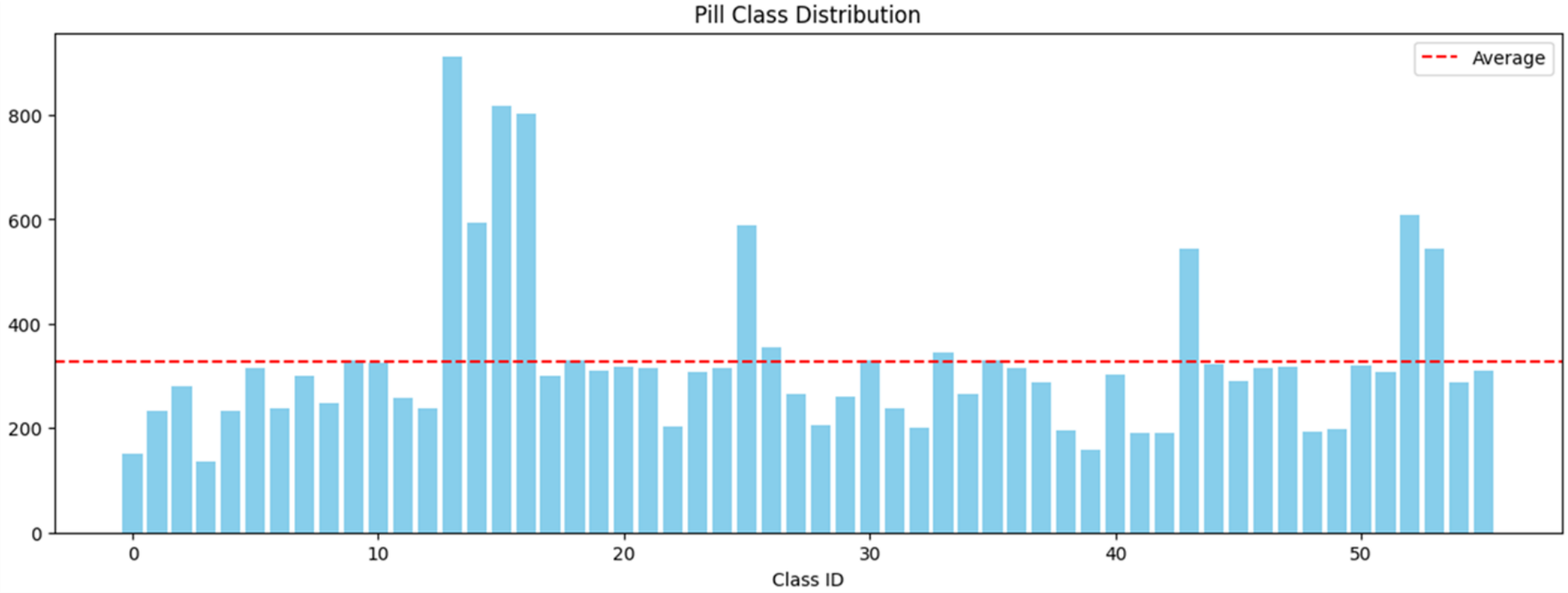
- 배경: 이미지 한 장에 여러 객체가 포함된 다중 객체 데이터 특성상, 무작위 삭제 시 소수 클래스까지 손실될 위험 존재.
- 로직
  소수 클래스(평균의 50%), 다수 클래스(평균의 150%), 그 외 클래스 이렇게 3개로 나눔.

  1. 소수 클래스 포함시 : 무조건 100% 보존
  2. 1에 포함되지 않으면서 다수 클래스가 존재 : 30%만 샘플링하여 보존
  3. 1과 2에 포함되지 않는 그 외 클래스로만 구성된 데이터 : 70%만 랜덤샘플링하여 보존.

해당 과정을 통해 다양성은 유지하면서 다수 클래스의 과도한 비중을 억제할 수 있었음.

데이터 문제 4) 클래스 불균형

두 전략의 적용 결과



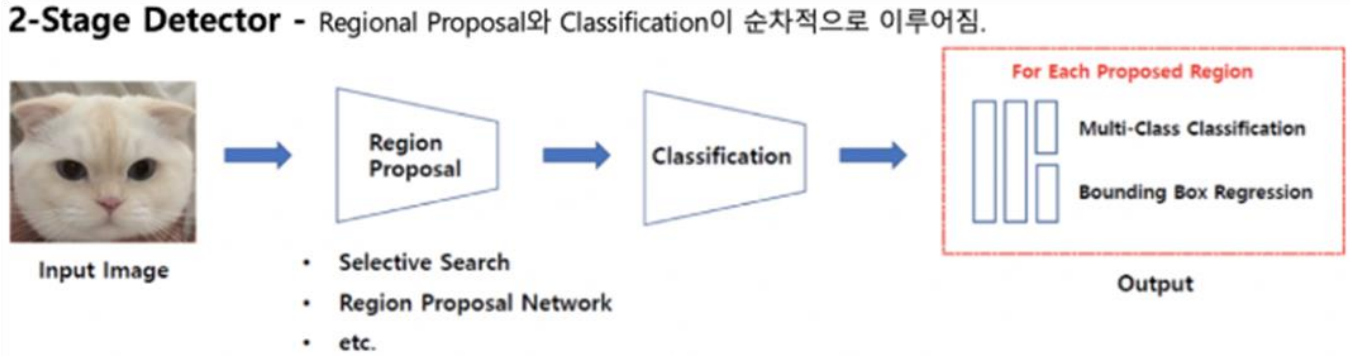
- 최대한 많은 클래스들을 평균값과 가깝게 했음.
- 최대 빈도 클래스와 최소 빈도 클래스간의 격차를 20배에서 8배로 감소시켰음

Faster R-CNN 실험

모델 설정 배경

팀의 주요 모델인 yolo 모델과의 비교군 형성

- 현재 yolo의 1-stage 모델을 모델 아키텍쳐 팀원이 제작중
- 2-stage 모델인 Faster R-CNN을 통해 1-stage와 2-stage의 성능 차이를 확인하기 위해 선정
- 2-stage 모델은 객체 후보 영역을 먼저 제안한 첫번째 stage와 해당 영역을 바탕으로 분류를 하는 두번째 stage를 따로 하는 모델이다.



Faster R-CNN 실험

모델 구축

- 입력 해상도 : 640 x 640 리사이징 적용
- 데이터 정규화 : /255 픽셀 스케일링으로 0~1사이로 조정
- 백본 네트워크 : Resnet-50 사용
- epoch = 15, learning rate = 0.0005, momentum = 0.9, weight_decay = 0.0005

- GPU는 3080을 사용하여 학습 (VRAM 사용량 7.8GB)

mAP50	mAP75	mAP50-95	Training Time (per epoch, Second)
0.921	0.884	0.821	530

Faster R-CNN 실험

모델 성능 최적화 및 하이퍼 파라미터 튜닝

- 입력 해상도 : LongestMaxSize(max_size=800) 로 원본 이미지 비율 유지
- 데이터 정규화 : Normalize(Imagenet mean , std) 로 Imagenet 기반 분포에 따른 정규화
- learning rate = 0.001 로 하이퍼파라미터 조정.

mAP50	mAP75	mAP50-95	Training Time (per epoch, AvgSecond)
0.9688	0.9638	0.9075	600

기존 이미지 데이터가 976 x 1280의 직사각형 형태의 비율을 가지고 있어 이미지 비율을 지키는 LongestMaxSize의 효과와 기존 리사이징보다 큰 크기에 의해 특징을 더 잘 잡아냈다 예측함

Faster R-CNN 실험

각 클래스 불균형 문제 해결과정의 효과 확인

Application	mAP50	mAP75	mAP50-95	Training Time (per epoch, Avg Second)
Augmentation	0.9642	0.9603	0.9138	890
Aug + sampling	0.9646	0.9657	0.9169	610

증강을 통해 늘어난 데이터 양에 의해 학습속도가 늘어났으며 mAP 50-95에서 성능 향상을 보임

샘플링을 통해 데이터 수를 다시 줄여 학습속도가 줄어들었으며 mAP 에서는 비슷한 성능을 보임.

Faster R-CNN 실험

학습중 전처리 추가

1. CLAHE 적용

알약의 각인, 테두리를 강조하는 효과를 주어 학습에서 알약의 특징을 더 뚜렷히 학습할 수 있도록 도움

2. Colorjitter

알약의 색상 정보는 보존하기 위해 hue 값을 제외하여 밝기, 대비, 채도를 변경하여 카메라나 조명 환경에 따른 변동성을 임의로 주도록 함.

두 변경사항 모두 원본 데이터 특징을 보존하기 위해 50%만 랜덤 적용하는 방식 사용

mAP50	mAP75	mAP50-95	Training Time (per epoch, AvgSecond)
0.9716	0.9696	0.9225	643

01 Project Manager

02 Data Engineer (R-CNN)

03 Model Architect

04 Experimentation Lead

1. 베이스라인 모델 선정

1) YOLOv8 vs YOLO26 (n, s, m)

Model	Precision	Recall	mAP50	mAP50-95	Training Time (h)
yolo26m	0.933	0.953	0.967	0.949	0.529
yolov8s	0.958	0.971	0.963	0.942	0.454

2) YOLOv7 vs YOLO26m

Model	Precision	Recall	mAP50	mAP50-95	Training Time (h)
yolov7	0.984	0.988	0.98	0.958	7.903 h / 20 epochs
yolo26m	0.971	0.978	0.98	0.972	1.880 h / 15 epochs

학습 효율 + mAP ⇨ yolo26m

2. 파이프라인 수정 - train/val 분할 방식

이미지 단위 무작위 분할

데이터 특성상 조명이나 각도가 조금씩 다른 유사 이미지가 train/val 세트에 섞여 들어갈 확률 증가

→ **데이터 누수 및 과적합 가능성**

알약 ID 조합별 분할

파일명에서 알약 조합 키를 추출하여 조합 단위 그룹으로 분할

K-**003544-004543-016548-027993**_0_2_0_2_90_000_200

K-**003544-004543-016548-027993**_0_2_0_2_75_000_200

⇒ **조합 ID:** K-003544-004543-016548-027993

2035장으로 학습 결과: mAP50-95가 0.009 차이로 유의미하진 않았다.

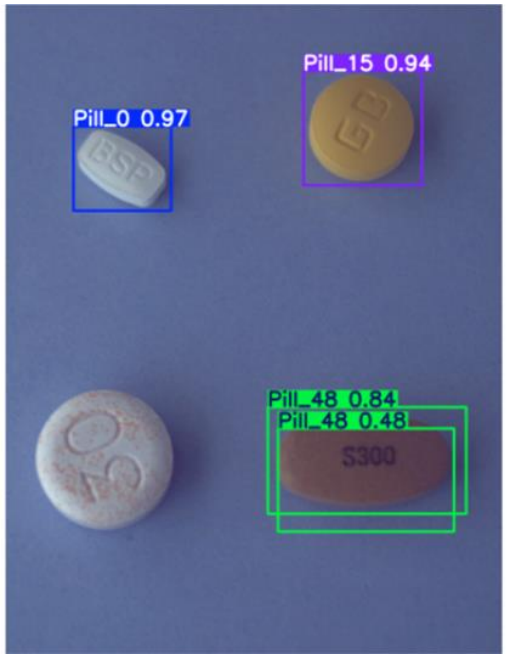
2. 파이프라인 수정 – 유효성 검사

bbox 리스트 길이 확인

하나의 객체가 [x, y, w, h] 형태인지 확인

bbox 길이 + 정규화

정규화된 좌표 범위(0~1) 추가 검사.
유효 객체가 하나라도 없으면 이미지 자체를 제외



3. 정제된 데이터로 학습

- **데이터셋** (7828장): 증강 + bbox 이상치 제거
- **epochs**: 20
- **imgsz**: 768
- **batch**: 16

Model	mAP50	mAP50-95	Training Time (h)
yolo26m	0.988	0.982	2.815

score: 98975

모델 분할

알약의 형태에 따라 전용 모델을 따로 학습하도록 함

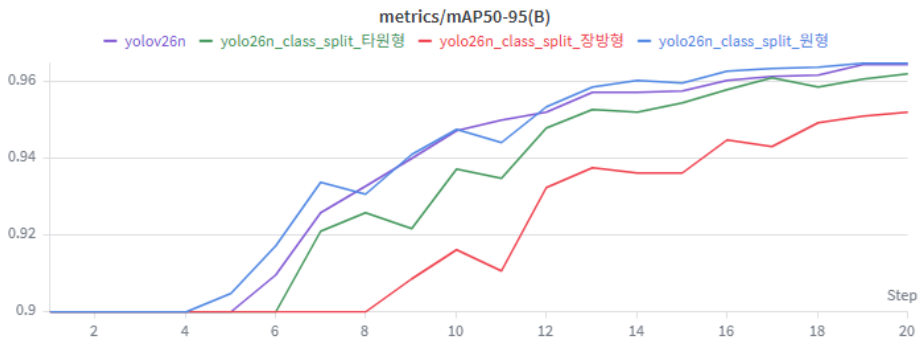
데이터 분석에서 원형, 장방형, 타원형, 팔각형, 육각형이 있는데 이 중 팔각, 육각형은 1개씩이므로 비슷한 형태의 원형으로 통합

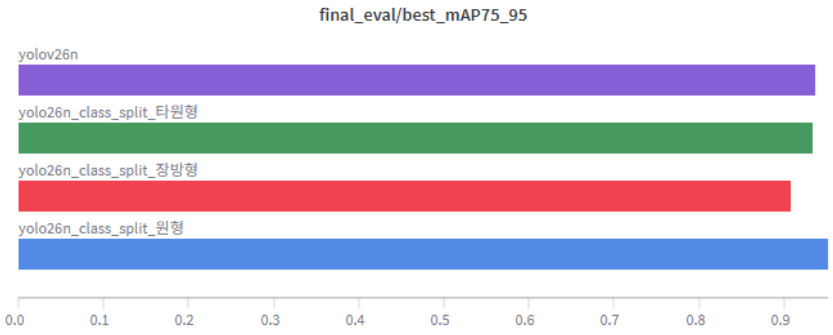
각 모양의 객체가 있는 데이터끼리 나눠서 학습.

Model	mAP50	mAP75	mAP50-95	Training Time (per epoch, Avg Second)
Circle	0.9961	0.994	0.9661	424
Rectangle	0.9994	0.9978	0.9485	290
Oval	0.9934	0.9896	0.9569	412

R-CNN의 형태별 결과

모델 분할





Score

0.97032 → 0.97792

시간관계상 YOLOv26n을 활용한 가설 검증 선행

기대효과 : 경량 모델에서의 성능 향상이 확인됨에 따라, 파라미터가 많은 YOLOv26m에 적용 시 기존 best 모델보다 성능 향상 기대

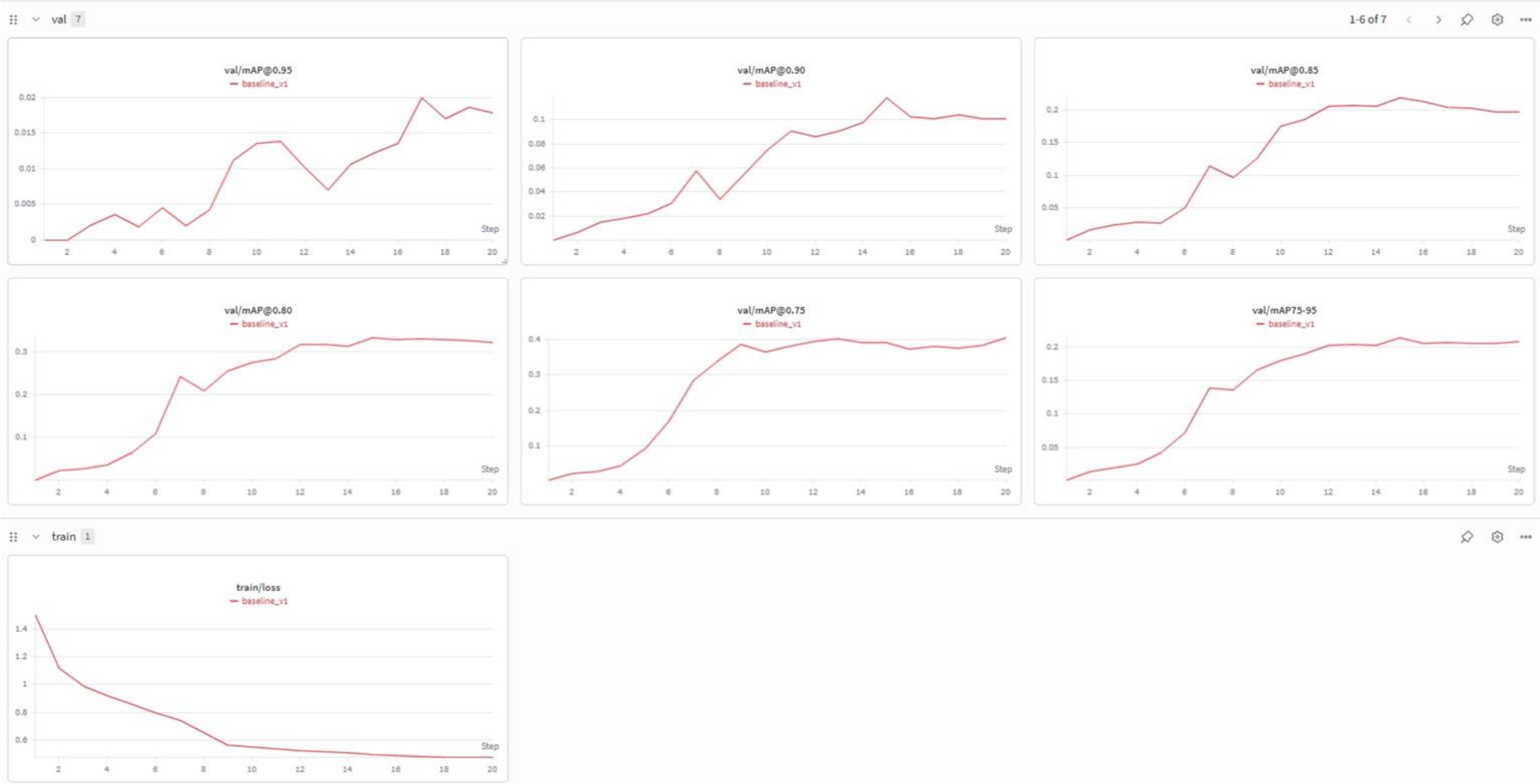
01 Project Manager

02 Data Engineer (R-CNN)

03 Model Architect

04 Experimentation Lead

1. wandb 연결


val 7
1-6 of 7
val/mAP@0.95
baseline_v1
val/mAP@0.90
baseline_v1
val/mAP@0.85
baseline_v1
val/mAP@0.80
baseline_v1
val/mAP@0.75
baseline_v1
val/mAP75-95
baseline_v1
Step
train 1
train/loss
baseline_v1

2. 평가 지표 구현 (mAP75-95)

수식

$$mAP75:95 = 1/N \sum_{t \in 0.75, 0.80, 0.85, 0.90, 0.95} AP_t$$

구현 과정

1. **IoU threshold**를 각각 **0.75 ~ 0.95**로 **0.05**씩 나누어 **AP** 계산
2. 결과 값을 클래스 개수 만큼 나누어 평균값 도출

구현 코드

```python
# ------------------------------
# IoU 계산
# ------------------------------
def compute_iou(box1, box2):
    """
    box format: [x1, y1, x2, y2]
    """
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter = max(0, x2 - x1) * max(0, y2 - y1)

    area1 = (box1[2]-box1[0]) * (box1[3]-box1[1])
    area2 = (box2[2]-box2[0]) * (box2[3]-box2[1])

    union = area1 + area2 - inter
    return inter / union if union != 0 else 0

# ----------------------------
# AP 계산
# ----------------------------
def compute_ap(recalls, precisions):
    recalls = np.concatenate(([0], recalls, [1]))
    precisions = np.concatenate(([0], precisions, [0]))

    for i in range(len(precisions)-1, 0, -1):
        precisions[i-1] = max(precisions[i-1], precisions[i])

    idx = np.where(recalls[1:] != recalls[:-1])[0]
    ap = np.sum((recalls[idx+1] - recalls[idx]) * precisions[idx+1])
    return ap
```

```python
# ----------------------------
# AP @ IoU threshold
# ----------------------------
def ap_at_iou(pred_boxes, pred_scores, gt_boxes, iou_threshold):

    sorted_idx = np.argsort(-pred_scores)
    pred_boxes = pred_boxes[sorted_idx]
    pred_scores = pred_scores[sorted_idx]

    tp = np.zeros(len(pred_boxes))
    fp = np.zeros(len(pred_boxes))
    matched = set()

    for i, pbox in enumerate(pred_boxes):
        best_iou = 0
        best_gt = -1

        for j, gt in enumerate(gt_boxes):
            iou = compute_iou(pbox, gt)
            if iou > best_iou:
                best_iou = iou
                best_gt = j

        if best_iou >= iou_threshold and best_gt not in matched:
            tp[i] = 1
            matched.add(best_gt)
        else:
            fp[i] = 1

    tp = np.cumsum(tp)
    fp = np.cumsum(fp)

    recalls = tp / len(gt_boxes)
    precisions = tp / (tp + fp + 1e-6)

    return compute_ap(recalls, precisions)
```

3. 전처리 성능 검증 실험

1. 배경
   a. 전처리 후 데이터셋의 학습 성능이 Raw 데이터셋보다 얼마만큼의 성능 향상이 이루어졌는지 실험

2. 실험결과
   a. 전체적으로 높은 성능 향상을 보임

[Kaggle Data] vs [전처리 후 데이터] 성능 비교

Data Set	mAP75	mAP80	mAP85	mAP90	mAP95	Candidates
Kaggle Data	0.5595	0.5599	0.5599	0.5599	0.4513	0.5331
Preprocessing Data	0.9869	0.9853	0.9838	0.9822	0.9330	0.9741

3. 전처리 성능 검증 실험

3-1. 성능 지표 시각화

a. 전처리 후 데이터 셋을 기준으로 학습시킨 결과가 더 높았음

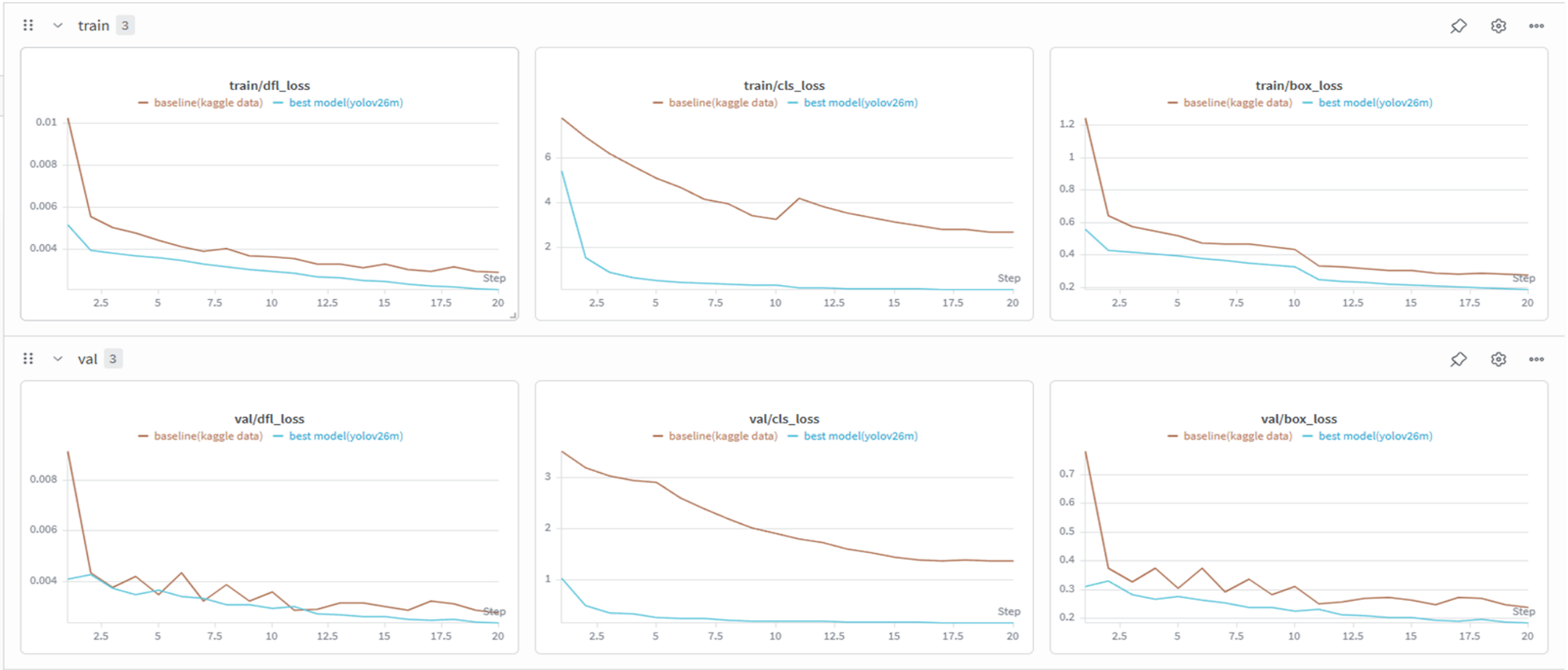
[Kaggle Data] vs [전처리 후 데이터] 성능 비교

3. 전처리 성능 검증 실험

3-2. 학습 지표 시각화

a. 전처리 후 데이터 셋을 기준으로 학습시킨 지표가 더 안정적임을 볼 수 있음

[Kaggle Data] vs [전처리 후 데이터] 학습 지표 비교



4. 하이퍼파라미터 튜닝

ultralytics

Ulyralytics: YOLO Model들을 간편하게 사용할 수 있게 도와주는 플랫폼

1. **Ulyralytics 플랫폼 선정 이유**
   a. **설치가 간단하며, 코드를 단순화 시킬 수 있다는 장점 존재.**
   b. **웬만한 기능들은 구현되어있어, 라이브러리에서 직접 가져다가 사용 가능.**

pip를 사용한 설치

CLI

```
pip install -U ultralytics
```

```
# Start training on your custom dataset
model.train(data="path/to/dataset.yaml", epochs=100, imgsz=640)
```

4. 하이퍼파라미터 튜닝

ultralytics

Ulyralytics: YOLO Model들을 간편하게 사용할 수 있게 도와주는 플랫폼

2. **Ulyralytics를 통한 자동화된 튜닝 (공식 문서 참조)**
   a. **자체적으로 제공하는 라이브러리를 통해 튜너가 하이퍼파라미터를 자동으로 변형하여 mAP를 최대화.**

```
from ultralytics import YOLO

# Initialize a YOLO26 model (using the 'nano' weight for speed)
model = YOLO("yolo26n.pt")

# Start tuning hyperparameters on the COCO8 dataset
# The tuner runs for 30 epochs per iteration, evolving parameters like lr0 and momentum
model.tune(data="coco8.yaml", epochs=30, iterations=100, optimizer="AdamW", plots=False)
```

4. 하이퍼파라미터 튜닝

ultralytics

3. 하이퍼파라미터 튜닝
   a. 전처리 된 데이터셋 中 일부만 사용
      (전체 데이터셋을 튜닝으로 사용시 20 epoch 기준 1 iteration 2시간)
   b. 해당 값을 바탕으로 학습 재진행

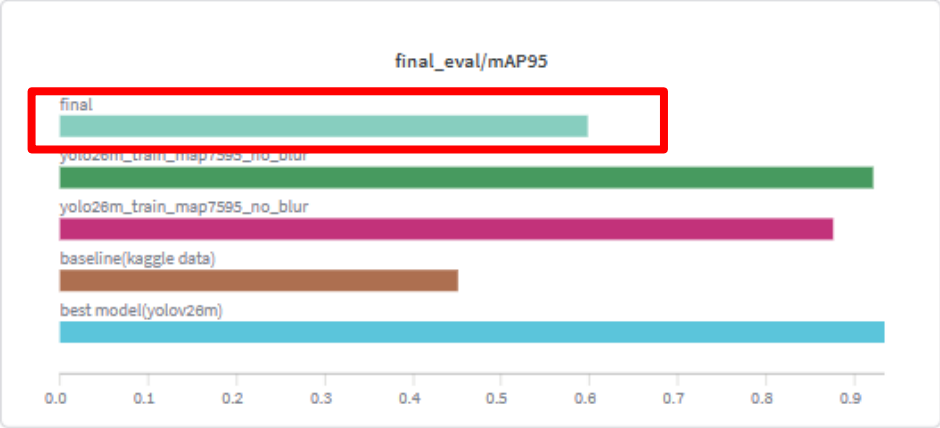
```
# tuned hyperparameters
    "lr0": 0.00794,
    "lrf": 0.0128,
    "momentum": 0.88528,
    "weight_decay": 0.00021,
    "warmup_epochs": 3.01527,
    "warmup_momentum": 0.95,
    # loss weights
    "box": 7.15031,
    "cls": 0.64925,
    "dfl": 1.64142,
    # color augmentation
    "hsv_h": 0.01081,
    "hsv_s": 0.76681,
    "hsv_v": 0.28674,
    # geometric augmentation
    "degrees": 1.0e-05,
    "translate": 0.10816,
    "scale": 0.37692,
    "shear": 0.00702,
    "perspective": 0.00041,
    # flip / channel augmentation
    "flipud": 0.00018,
    "fliplr": 0.6106,
    "bgr": 0.00136,
    # mix augmentation
    "mosaic": 0.99492,
    "mixup": 0.006
```

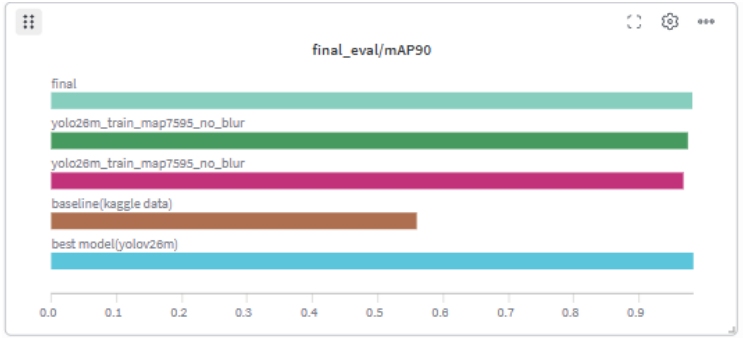
4. 하이퍼파라미터 튜닝

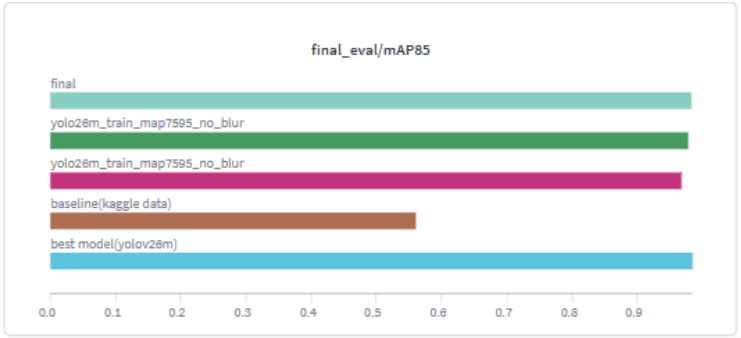
ultralytics

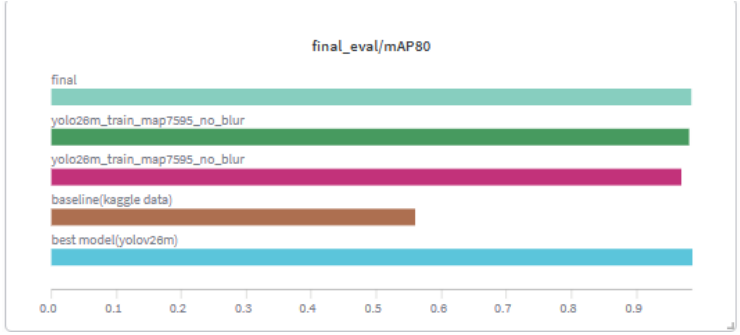
4-1. 성능 비교

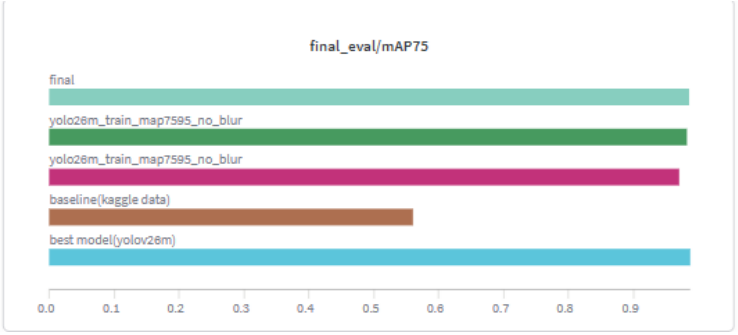
a. mAP95를 제외한 나머지 성능인 좋았으나, 95만 다른 모델에 비해 성능이 확연히 떨어짐을 관찰.











4. 하이퍼파라미터 튜닝

ultralytics

4-2. 원인 분석

a. IoU 75~90은 상대적으로 높은 점수를 받았으나, 95는 상대적으로 낮은 점수를 받음

b. 데이터셋의 문제인지 점검

   i. 이전 실험에서 쓰인 데이터셋과 동일한 데이터를 쓴 것으로 확인되어 해당 경우의 수는 제외.

c. 하이퍼 파라미터 문제인지 점검

   i. 하이퍼파라미터를 적용하기 전 모델들이 이상이 없었으므로 파라미터 값을 분석

   ii. 해상도 및 증강과 관련된 파라미터들을 발견할 수 있었다.

d. 가설

   i. 해상도와 관련된 값이 훈련에 지장을 줄 것이라 가정

   ii. 이에 해당 augmentation들을 제거 후 훈련 재게

```
# color augmentation
"hsv_h": 0.01081,
"hsv_s": 0.76681,
"hsv_v": 0.28674,
# geometric augmentation
"degrees": 1.0e-05,
"translate": 0.10816,
"scale": 0.37692,
"shear": 0.00702,
"perspective": 0.00041,
# flip / channel augmentation
"flipud": 0.00018,
"fliplr": 0.6106,
"bgr": 0.00136,
# mix augmentation
"mosaic": 0.99492,
"mixup": 0.006
```

4. 하이퍼파라미터 튜닝

ultralytics

4-2. 수정 결과

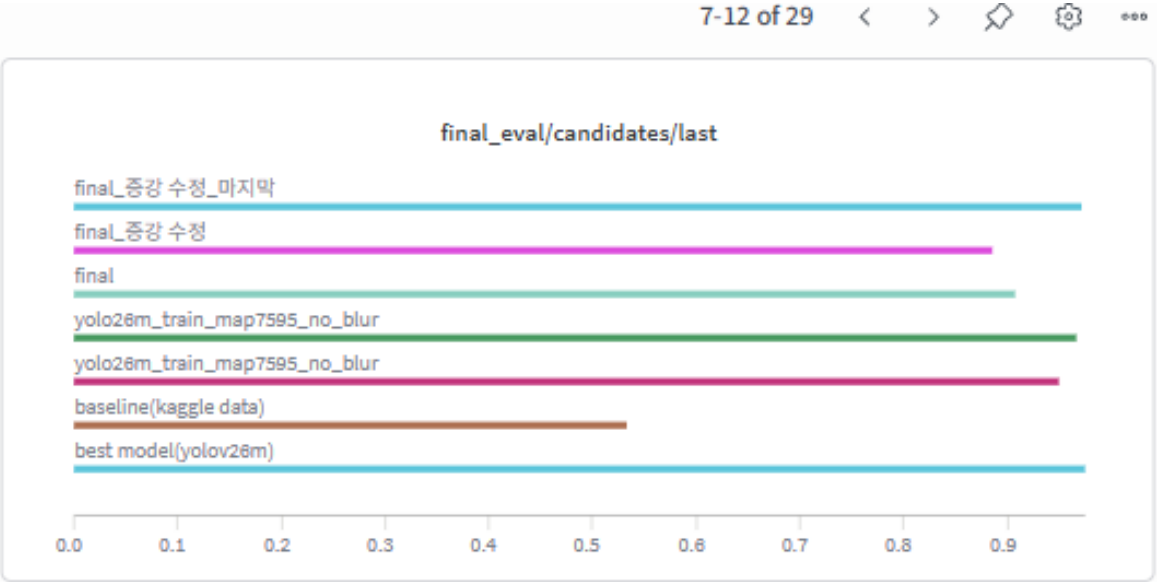
a. 문제가 되는 하이퍼파라미터 제거를 통해 성능이 상승함을 관찰할 수 있었음.

b. 그러나 임의로 선정한 하이퍼파라미터에 비해 성능이 0.003의 근소한 차이로 낮음을 확인

   i. 전체 데이터셋으로 튜닝을 진행했더라도 근소하게 성능 향상이 있을 것으로 예상

   ii. 모델 자체의 성능이 뛰어나기 때문에 시간상 임의로 선정한 하이퍼파라미터를 채택 후 실험 종료

Model	Precision
best model	0.974
tuning model	0.971



감사합니다